

Document Question Answering System Using Retrieval-Augmented Generation

Objective

Build an end-to-end RAG-based Document QA system that lets users upload PDF or Excel files and then ask questions to get answers from the documents, with source citations.

General Requirements

- The backend must be implemented using **Django** and **Django REST Framework**.
 - The project must use a relational database (e.g., PostgreSQL, MySQL, etc.).
 - The project must be runnable using Docker.
-

Functional Requirements

1 User Authentication

- Users must register and log in.
- Only authenticated users can upload files or ask questions.
- Each user must only be able to see and access their own documents.
- Users must NOT have access to other users' files or data.

You may use Django authentication or token-based authentication (JWT).

2 Document Upload

Create an API endpoint to upload:

- PDF files
- Excel files

When a file is uploaded:

- Extract text
- Split into chunks (with overlap)
- Generate embeddings
- Store embeddings in DB
- Store metadata (file name, page number if available, chunk index)

Each document must be associated with the authenticated user who uploaded it.

3 List User Documents

Create an API endpoint that allows a logged-in user to:

- View a list of their uploaded documents
- See basic information like `file_name`, `file_type`, and `upload_date`
- Search documents by keyword (e.g., partial match on file name)
- Filter documents using query parameters, such as:
 - `file_type` (e.g., pdf)
 - upload date range (`uploaded_after`, `uploaded_before`)
- Support sorting (e.g., newest first / oldest first)
- Support pagination for large lists

Users must only see their own documents.

4 Question Answering

Create an API endpoint where the authenticated user can:

- Send a question
- Optionally specify a document ID

The system must:

1. Convert the question into an embedding
 2. Retrieve the most relevant chunks of files (restricted to the user's documents)
 3. Generate an answer based only on that context
 4. Return the answer with source references
-

5 Track User Question Activity

The system must:

- Store each question asked by a user
- Associate each question with:
 - `user_id`
 - `document_id` (if specified)
 - `timestamp`
- Store the generated answer
- Store the response status (success / no answer / error)
- Store response time (in milliseconds)
- Maintain a counter of how many questions each user has asked
- Provide an API endpoint to retrieve:

- Total number of questions asked by the user
- Recent question history (optional)

6 Response Format

The response must include:

- The final answer
- A list of sources used to generate the answer

Example:

```
{
  "answer": "According to the document, customer satisfaction improved significantly in 2023 due to faster response times and expanded support coverage.",
  "sources": [
    {
      "file_name": "annual_report_2023.pdf",
      "page": 8,
      "chunk_index": 4,
      "excerpt": "Customer satisfaction improved in 2023 as response times decreased and support hours were expanded."
    }
  ]
}
```

If the system does not find enough information, it should return a clear message saying that the answer is not available in the documents.

Bonus (Optional)

You may implement any of the following:

- **Async processing**
Process document processing (text extraction, chunking, embedding) in a background job instead of handling it during the upload request.
- **Basic automated tests**
Add automated tests to verify core functionality such as authentication, document upload, question answering, and user data isolation.
- **Metadata filtering**
Allow searching to be restricted using metadata, such as limiting search to specific documents or specific page ranges.

- **Reranking**

After retrieving the initial top-k candidates using vector similarity, apply a second-stage relevance model (e.g., cross-encoder, LLM scoring, or pairwise ranking) to compute a more accurate relevance score between the question and each chunk, then select the highest-ranked chunks for the final prompt.

- **Streaming responses**

Return the generated answer incrementally (token-by-token or chunk-by-chunk) instead of waiting for the full response to complete.